Full Stack and Infrastructure Engineer Interview Preparation

Prepared for Muhammad Awais | Role in Al Rawdah, Abu Dhabi, UAE

Your main interview message: you can connect backend engineering, Azure deployment, and production operations for an AI platform. This guide contains 200 questions with sample answers, system design and troubleshooting drills, practical exercises, and a seven-day preparation plan.

Practice answers aloud for 60 to 90 seconds, then expand only when asked. Personalize every project story with details you can defend. Experience answers draw on your CV; design answers describe what you would do. Cost governance is a preparation priority because the CV does not establish ownership of budgets or quantified savings.

1 Tell me about yourself

I am a Senior Full Stack Engineer with more than eight years of experience delivering enterprise applications and cloud platforms. My backend work includes ASP.NET Core, Node.js and TypeScript, and Python FastAPI. At USS IT Consultants, contracted to 7X/EPG, my work includes AI-enabled services, a production RAG pipeline, infrastructure automation, and operational support. Previously, I worked on logistics at GAC, Azure transformation at Merik, and airline systems at Calrom. This role interests me because it combines backend delivery with the deployment, observability, and reliability responsibilities I have handled in production.

2 Why are you a good fit for this role

The role needs someone who can build APIs and keep the platform dependable after release. My CV includes AKS and Azure Container Apps, Terraform and Bicep, Azure DevOps and GitHub Actions, and incident response using Azure Monitor and Application Insights. I also understand the application side of AI through Azure OpenAI, retrieval, and Python inference services. I would use that combination to help AI and frontend engineers deliver features with deployment controls, measurable service health, and clear operational ownership.

3 What would you prioritize in the first month

First, I would map the services, dependencies, production access, current costs, and incident history. I would verify that a failed release can be reversed and that backups can actually be restored. Next, I would agree on service objectives and identify gaps in alerts, secret handling, and infrastructure automation. By the end of the month, I would aim to deliver one measurable improvement, such as a safer release path or an actionable dashboard, while documenting a prioritized reliability and cost backlog.

4 How should you handle an unfamiliar technology

I would state my exposure accurately, connect it to a relevant concept I know, and describe how I would validate a solution. For example: I have production experience with Azure infrastructure and monitoring, but my CV does not document ownership of cloud budgets. I can explain a cost-control design and would validate it with the billing owner before applying production spending controls. This is stronger than claiming a project outcome I cannot substantiate.

Question Index and Study Order

Start with questions 1 to 48 for the main role discussion. Questions 49 to 200 provide focused follow-up practice. Each answer is an outline to explain aloud and expand with evidence or tradeoffs; do not memorize unsupported experience claims.

Core preparation

1-8 Introduction and project evidence; 9-12 Azure design; 13-16 APIs; 17-20 delivery and IaC; 21-24 containers; 25-28 observability; 29-32 cost; 33-36 security; 37-40 AI operations; 41-44 incidents; 45-48 behavioral and recruiter questions.

Extended question bank

49-52 Azure Compute Choices; 53-56 Azure Networking

57-60 Identity and Access Operations; 61-64 Storage and Document Handling

65-68 Database Selection and Data Modeling; 69-72 Database Transactions and Concurrency

73-76 Database Performance; 77-80 Caching and Redis

81-84 Messaging Service Selection; 85-88 Reliable Message Processing

89-92 Microservice Resilience; 93-96 API Contracts and Compatibility

97-100 API Security and Traffic Controls; 101-104 Python Fundamentals

105-108 FastAPI and Python Service Lifecycle; 109-112 Node js and TypeScript

113-116 ASP NET Core and C Sharp; 117-120 Frontend and Backend Collaboration

121-124 CI Pipeline Engineering; 125-128 Release Strategies

129-132 Terraform Practice; 133-136 Bicep and Azure Governance

137-140 Docker Operations; 141-144 Kubernetes Networking and Configuration

145-148 Kubernetes Capacity and Scheduling; 149-152 Kubernetes Lifecycle and GitOps

153-156 Telemetry Design; 157-160 KQL Dashboards and Alerting

161-164 SRE and Operational Priorities; 165-168 Recovery Backups and Continuity

169-172 Infrastructure Cost Engineering; 173-176 LLM Cost Controls and Fairness

177-180 RAG Ingestion and Retrieval Quality; 181-184 AI Evaluation and Quality

185-188 Agent Orchestration and State; 189-192 Advanced Incident Scenarios

193-196 Coding and Design Follow Ups; 197-200 Leadership and Professional Judgment

How to rehearse

For each answer, explain why you chose that approach, one failure case, one alternative, and how you would verify it. Mark questions as confident, needs practice, or new. Use the exercises and mock interview at the end to test recall.

Project Stories and Evidence

Use Situation, Task, Action, Result. Be specific about your contribution and what the team delivered. Bring a simple architecture sketch for your two strongest projects.

5 Describe your AI platform work

At 7X/EPG, the platform combined content services, event processing, and ML inference. My CV describes a backend using .NET services, Node.js NestJS on Azure Functions, and Python FastAPI, with Azure Container Apps and Kubernetes deployment. The retrieval flow used Azure OpenAI, LangChain or Semantic Kernel, Azure AI Search, and pgvector. I would explain my own component ownership, how a request moved through the services, and how we monitored failures. The documented retrieval result was sub-two-second responses; I would clarify the measurement conditions rather than present that as an unconditional SLA.

Prepare follow-ups: Why these three languages? Which service did you own? What was the slowest dependency? How were prompts and retrieval evaluated?

6 Tell me about an infrastructure improvement

My current-role CV records infrastructure provisioning with Terraform and Azure DevOps, including AI service dependencies, in under five minutes. I would frame the problem as repeatable environment creation, explain how modules and environment configuration were organized, and describe validation before deployment. The result I can cite is faster provisioning. I would be ready to define which resources and pipeline stages the five-minute figure includes and separate that timing from application startup, data migration, or manual approval time.

7 Describe a reliability improvement

At GAC, my responsibilities included investigating API, container, messaging, and deployment incidents, executing rollback or recovery, and improving runbooks and alerts after RCA. For the interview, I would choose one real incident and state its symptom, impact, and my action. I would explain how I identified the failing dependency, restored service, and changed the system afterward. My CV supports this responsibility, but it does not give a specific outage timeline, so I would supply only details I remember accurately.

8 Explain the metrics on your CV

The CV records 200K+ daily events, 99.99% message delivery reliability, and 90%+ automated test coverage in the current platform. These measure different things. Event count describes throughput volume; message reliability needs a defined denominator, time window, and treatment of retries; coverage measures exercised code and does not prove correctness. I would explain where the measurements came from and my contribution. I would not relabel message reliability as API availability or claim that the coverage percentage guarantees the system has no defects.

Before the interview, record: exact service owned, workload size, measurement window, dashboard or test report, one failure mode, one tradeoff, and what you would change today. Confirm certificate status and employer dates before repeating them.

Azure Architecture and Environment Design

9 How would you design an Azure AI agents platform

I would first clarify tenant isolation, request volume, response time, data sensitivity, and whether work is interactive or asynchronous. A starting design is an authenticated API behind API Management, an orchestrator, and queue-backed workers for longer tasks. State belongs in a durable database, documents in storage, and retrieval in a search or vector service. I would use Azure OpenAI for model calls, workload identities for resource access, and tracing across the API, agent run, and tools. I would start with the simplest orchestration that meets the workflow and add services only when there is a clear operational reason.

10 When would you choose AKS or Container Apps

I would choose Container Apps when managed container hosting and scaling satisfy the workload and we want to reduce cluster operations. I would choose AKS when we need Kubernetes-level control, specialized scheduling, operators, or detailed networking and platform extensions. The decision includes team capability, workload behavior, security requirements, and total operating cost. My CV includes both environments. I would compare a representative workload and its operational requirements instead of assuming Kubernetes is always the right answer.

11 How do you separate development test and production

I would use deliberate environment boundaries for identity, configuration, data, deployment permissions, and resource ownership. Production would have restricted access and controlled release paths. Reusable infrastructure modules would keep environments consistent while parameter files hold environment differences. Nonproduction would use synthetic or appropriately sanitized data. I would separate production from nonproduction subscriptions when governance requires it, and verify that service identities cannot cross the boundary. Naming, tags, ownership, and cost reporting would be applied consistently.

12 How would you design availability and recovery

I would agree on a service-level objective, recovery time objective, and recovery point objective before adding redundancy. I would remove single points of failure within the chosen region, use appropriate database backup and replication, and ensure retries and timeouts prevent dependency failures from cascading. If regional recovery is required, I would design the data and identity failover as well as compute. I would test restoration and failover, because replicas and backup configuration alone do not demonstrate recovery. The final design depends on acceptable downtime, data loss, and budget.

Follow-ups: How do private endpoints resolve DNS? How do you rotate credentials? Which dependency limits scaling? Explain that availability zones and a second region solve different failure cases.

Backend APIs and Distributed Systems

13 How would you design an API for agent runs

I would expose a create-run endpoint, a run-status endpoint, and cancellation where the workflow supports it. A long-running request would return HTTP 202 with a run identifier and status location. The API would validate input, authorize the tenant and resource, and persist accepted work before dispatch. Clients could poll or receive streamed progress. I would define versioned request and response contracts, consistent errors, request limits, and a correlation identifier. The API should distinguish accepted, running, succeeded, failed, and canceled states.

14 How do you make a retry safe

For an operation that can create side effects, I would accept an idempotency key scoped to the tenant and operation. I would atomically store the key with a request hash and operation state under a uniqueness constraint. A repeated matching request returns the same run or result; a conflicting payload returns a defined error. I would decide key expiration and behavior for in-progress or failed operations. Retry logic alone is insufficient because a timeout may happen after the original action has already succeeded.

15 What differs between Python async and Node.js concurrency

Both can handle many I/O waits efficiently, but asynchronous syntax does not make blocking work disappear. In Python, I would use nonblocking clients in an async endpoint and move CPU-heavy work to a suitable process or worker service. In Node.js, I would avoid blocking the event loop and use worker threads or separate workers for CPU-heavy tasks. In either stack, I would limit connection pools and concurrency, set deadlines, and measure queueing. Adding unlimited concurrency can overwhelm databases or model quotas.

16 How do you keep events and database state consistent

I would use a transactional outbox when a database update must reliably lead to an event. The transaction writes the business record and an outbox record together. A dispatcher publishes pending records and records progress; consumers deduplicate because publication or delivery can repeat. I would monitor oldest unpublished age, retries, and dead letters. For a multistep business workflow, I would define compensating actions where possible. I would avoid promising end-to-end exactly-once side effects without explaining the boundaries and deduplication mechanism.

Also rehearse: cursor pagination, schema validation, API versioning, authentication versus authorization, database indexes, connection pool exhaustion, and HTTP 401, 403, 409, 429, 502, 503, and 504.

CI CD and Infrastructure as Code

17 Describe a production delivery pipeline

I would run linting, unit and integration tests, dependency and container scanning, then build one immutable artifact and publish it to the registry. Infrastructure changes would have a reviewed plan or preview. The same application artifact would progress through test and production, with environment configuration injected separately. Before rollout, I would check migration compatibility and readiness. After rollout, I would verify errors, latency, and key business behavior. Azure DevOps and GitHub Actions are both represented in my CV; I would explain the actual pipeline I worked on rather than only name the tooling.

18 How would you make deployments reversible

I would retain the previous application version and define the health signals that stop a rollout. A small initial deployment would expand only after its health is acceptable. Recovery could switch traffic back or deploy a known-good image. I would treat database changes separately, using compatible schema expansion before removing old fields later. Some data mutations require a forward fix or restoration rather than a simple application rollback. I would test recovery steps and make ownership clear before production release. [S1]

19 When would you use Terraform or Bicep

Bicep is attractive for Azure-focused resource definitions and integration with Azure Resource Manager. Terraform is useful when a common workflow spans Azure and other providers. For Terraform, I would protect remote state, use backend-supported locking, restrict access, and review plans. For Bicep, I would review what-if output and understand its limitations. For either, I would pin dependencies appropriately, structure reusable modules, and manage secrets outside ordinary configuration files. My CV includes Terraform and Bicep, so I would discuss a real example of each.

20 How do you handle infrastructure drift

I would detect drift through scheduled comparison and investigate why it occurred before changing production. An emergency manual change should be recorded, reviewed, and represented in code if it must remain. If drift should be removed, I would assess impact and apply a controlled correction. I would also check deployment access, ownership, and policy to reduce recurrence. Blindly applying infrastructure code can remove a critical emergency fix; permanently accepting unmanaged drift undermines reproducibility.

Follow-ups: immutable image tags versus digests; secret-free pipeline identity; required reviews; artifact promotion; smoke tests; database expand-and-contract; emergency change reconciliation. Reference: Bicep what-if [S2].

Containers Kubernetes and Scaling

21 What makes a production container image good

I would use a trusted minimal base, build in stages, install only runtime dependencies, and run as a non-root user where supported. The image should not contain credentials or environment-specific settings. I would scan it, record its origin, and deploy an immutable digest. The process should handle termination signals, stop accepting new work, and finish or safely checkpoint current work. I would also check the actual memory profile and writable filesystem needs before enforcing runtime restrictions.

22 Explain readiness liveness and startup probes

Readiness determines whether a pod should receive traffic. Liveness detects a condition that justifies restarting the container. A startup probe protects a slow-starting application before readiness and liveness checks proceed. I would make liveness conservative; a database outage should not cause every healthy application process to restart. Readiness should reflect whether the service can perform its intended work. Probe thresholds and intervals need to match observed startup and failure behavior. [S3]

23 How would you scale workers and APIs

For interactive APIs, I would consider request concurrency, CPU or memory, and latency. For workers, queue depth and the age of the oldest item are often better indicators. Pod scaling and node scaling solve different capacity problems. I would set resource requests and limits based on measurements, define minimum and maximum capacity, and validate that the database and model endpoint can handle the resulting concurrency. More workers can increase throttling and cost if the downstream dependency is already saturated.

24 A pod is failing how do you investigate

I would inspect status, events, current and previous logs, and the last deployment change. Pending can indicate scheduling, capacity, or volume issues. ImagePullBackOff points toward registry access or image references. CrashLoopBackOff means I need the crash cause, such as startup errors, configuration, or failed liveness. OOMKilled suggests examining memory limits and actual allocation. I would restore service with a safe mitigation and then address the cause. I would not increase limits or restart pods repeatedly without understanding the failure.

`Practice: kubectl get pods; kubectl describe pod <pod>; kubectl logs <pod> --previous; kubectl get events; kubectl rollout status deployment/<name>`

Follow-ups: disruption budgets, availability-zone placement, rolling-update surge, graceful shutdown, resource limits, service discovery, and why an HPA cannot overcome a model quota.

Observability Reliability and Incident Response

25 What would you monitor on this platform

I would monitor the user journey and its dependencies: run completion and failure rates, p95 and p99 latency, queue age, tool failures, database saturation, and model throttling. For AI workloads, I would add tokens and estimated cost per completed task, repeated tool calls, and evaluation results. Traces would connect API requests to run IDs, model calls, and tool operations, with sensitive content excluded from default logs. My CV includes Azure Monitor, Application Insights, OpenTelemetry, Grafana, Prometheus, and Loki.

26 What are an SLI SLO and error budget

An SLI is the measured indicator, such as the proportion of valid requests completed successfully. An SLO is its target over a specified period. The error budget is the allowed unreliability implied by that target. For example, a 99.9% request-success target permits 0.1% unsuccessful eligible requests; we must define eligibility and measurement. I would use budget consumption to guide release risk and reliability work. I would distinguish an internal objective from a contractual SLA and avoid adopting a target without business agreement.

27 How do you respond to a severe production incident

I would confirm user impact, establish ownership and a communication channel, and capture the incident start time. I would inspect recent changes and current signals, then choose the safest mitigation, such as stopping a rollout or disabling a failing feature. I would keep investigation and restoration coordinated, verify recovery from user-facing signals, and communicate what remains affected. Afterward, I would preserve the timeline, determine contributing causes, and assign prevention work. My documented on-call experience includes rollback, traffic mitigation, and RCA.

28 How do you reduce noisy alerts

I would review whether each alert identifies an actionable user impact or an imminent risk with a known response. I would use appropriate duration windows, group duplicates, route alerts to owners, and attach runbooks. A temporary CPU spike should not automatically page the team if the service remains healthy. I would also detect missing telemetry so silence is not mistaken for health. After incidents, I would review false positives and missed detections, then adjust signals and thresholds based on evidence.

Rehearse an incident update: affected users and functions, current mitigation, next investigation step, and next update time. Do not present an unverified suspected cause as a confirmed root cause.

Cloud and LLM Cost Management

Use proposed-action language for these answers unless you can add a verified personal example. Do not invent a savings percentage, budget size, or finance responsibility.

29 How would you manage Azure spending

I would establish a spending baseline by environment, service, and owner, then identify major cost drivers and unused resources. I would configure actual and forecast budget alerts at suitable scopes and assign response owners. Azure budgets notify; they do not automatically stop resources, and billing data is delayed, so they cannot serve as an immediate spending cutoff. I would use runtime quotas and an application usage ledger for fast controls, with billing reconciliation afterward. Any automatic restriction of production would need agreed service behavior. [S4]

30 How would you reduce LLM cost without losing quality

I would measure cost per successfully completed task, not only cost per call. Then I would test shorter retrieved context, appropriate output limits, reduced unnecessary retries, and routing simpler tasks to an adequate model. I would consider caching only where tenant permissions and freshness allow it. Agent step limits and tool-call limits prevent runaway loops. Each change needs a representative evaluation set and latency measurements, because a cheaper call can cause more failures, retries, or escalations. I would use current deployment pricing when estimating impact.

31 Explain a token cost calculation

For token-billed calls: cost = input_tokens / 1,000,000 x input_rate + output_tokens / 1,000,000 x output_rate. Hypothetical practice rates of USD 2 per million input tokens and USD 8 per million output tokens give USD 0.012 for 2,000 input and 1,000 output tokens. At 10,000 such calls, that is USD 120. These are invented exercise rates, not a model price quote. A full task estimate also includes extra model calls, embeddings, search, compute, storage, telemetry, and retries; some deployment types use different billing models.

32 How would you investigate a sudden cost spike

I would compare cost and usage by time, deployment, tenant, and workload. I would check recent releases, prompt growth, model-routing changes, traffic volume, repeated tool calls, retries, background jobs, and possible credential misuse. I would contain the driver using targeted limits or a feature switch while preserving critical service. Then I would validate the effect in runtime usage metrics and reconcile billing later. For infrastructure, I would also inspect excess replicas, idle capacity, telemetry ingestion, and unexpected data transfer.

Pragmatic tradeoffs: right-size before commitments; schedule nonproduction shutdown only where safe; reduce log retention by need; set tenant limits; distinguish forecast alerts from immediate enforcement. Prioritize this page in rehearsal.

Identity Security and DevSecOps

33 How would you manage secrets and identities

I would prefer workload identities over embedded credentials and grant only the required resource permissions. For AKS, workload identity can federate the Kubernetes service account with an Azure identity. Secrets that remain necessary would be stored in Key Vault with restricted access, rotation, and an audit trail. Pipeline access would use federation where supported. I would ensure local developer access, production runtime access, and deployment access have different scopes. My CV documents Key Vault and Azure AD use; I would describe workload federation as a design approach unless I can verify personal implementation. [S5]

34 How do you isolate tenants

I would derive tenant context from authenticated claims and enforce authorization in each data access path. A client-supplied tenant ID is not sufficient proof of access. Tenant identity must carry through API requests, queued jobs, caches, retrieval filters, and tool calls. I would choose shared or separate databases and compute based on isolation requirements, operational cost, and noisy-neighbor risk. I would test cross-tenant access attempts and avoid caching sensitive results under keys that omit the security context.

35 How would you secure agent tools

I would treat model-generated tool arguments as untrusted input. Each tool would have a narrow contract, validated arguments, independent authorization, and restricted network and data access. Retrieved text would not be allowed to override application permissions. Sensitive actions would have an approval or other explicit authorization boundary and a useful audit record. I would limit steps and external destinations, redact sensitive logs, and test malicious documents. Prompt instructions help guide behavior, but enforcement belongs in application and infrastructure controls.

36 What does DevSecOps mean in your delivery process

I would integrate security checks into ordinary delivery: secret detection, dependency and image scanning, infrastructure review, least-privilege deployment access, and tracked remediation. Critical findings need an agreed release policy and a time-bound exception process where necessary. Runtime controls remain important because a clean scan does not guarantee secure behavior. At GAC, my CV includes automated security scanning and container image signing. I would be ready to explain the actual checks, what happened on failure, and who reviewed exceptions.

Follow-ups: authentication versus authorization; Azure RBAC versus Kubernetes RBAC; private endpoint DNS; certificate rotation; audit logging without exposing prompts or credentials; containment and rotation after a leaked credential.

AI Platform Operations and Retrieval

37 Explain RAG and how you would operate it

Retrieval-augmented generation retrieves relevant information and includes it in the model context. I would explain ingestion, parsing, chunking, indexing, permission filtering, retrieval, and generation separately. Operational checks include freshness, retrieval failures, index updates, and latency at each stage. Evaluation should measure whether the right evidence was retrieved and whether the answer is supported, not only whether it sounds fluent. My current-role CV includes Azure AI Search and pgvector with Azure OpenAI, so I would anchor the answer in that actual pipeline.

38 How would you handle Azure OpenAI throttling

I would inspect throttling responses and quota signals, control concurrency, and smooth bursts before the model call. Retry handling would respect the returned retry delay when available and use bounded backoff with jitter otherwise. I would account for existing SDK retries so retries do not multiply across layers. Quota varies by region, model, and deployment type; the context window is a separate limit. I would monitor throttling and queue delay and return a clear overload response if the task deadline cannot be met. [S6]

39 How do you keep an agent from running forever

I would enforce maximum run duration, model calls, tool calls, and token or cost allowances in the orchestrator. Progress would be persisted so a worker restart does not blindly repeat side effects. The run state would record completed steps and cancellation requests. Before each expensive or sensitive step, I would check remaining allowance and authorization. When a limit is reached, the system should stop safely and explain the outcome. A prompt requesting brevity does not provide a reliable operational limit.

40 How would you release a model or prompt change

I would version model deployment, prompt, tools, retrieval configuration, and evaluation data. Before production, I would run representative and adversarial cases, including failure and permission tests. I would compare quality, latency, tool-call behavior, and cost per completed task against the baseline. A staged rollout would allow reversal to a known-good configuration. Live monitoring should detect quality drift and changes in user traffic. I would agree on acceptable tradeoffs with AI and product engineers rather than treat a prompt edit as a risk-free text change.

Understand agent orchestration choices: fixed sequence for predictable stages, controlled parallelism for independent work, and handoff only when routing adds value. More agents add state, coordination, quota, and debugging overhead. [S7]

System Design Whiteboard Practice

Design a secure document analysis agent on Azure

Practice prompt: enterprise users upload documents and ask an agent to extract information, retrieve relevant evidence, and produce a report. Some jobs take minutes. Explain your design in 20 minutes and reserve five minutes for tradeoffs.

A strong opening

I would first ask about peak users, document size, job duration, tenant isolation, retention, required accuracy, expected completion time, and recovery objectives. I would distinguish an interactive answer from an asynchronous report. I would also confirm where data and model processing are permitted before choosing regions or deployments.

```
User -> API Management -> Run API -> Durable run database
Run API -> Outbox dispatcher -> Queue -> Agent workers
Workers -> Document storage / Retrieval / Model / Approved tools
All stages -> Correlated telemetry and per-run usage ledger
```

Explain the flow

1. Authenticate the user, authorize the tenant, and validate file and request limits. Accept a run with an idempotency key. Persist run state and dispatch intent atomically.
2. Validate uploads, ingest documents, and build permission-aware retrieval. Workers process durable jobs with bounded concurrency.
3. Each agent step checks authorization and allowance, calls the model or tool with a deadline, and records progress.
4. Save the result and completion state; provide status polling or controlled streaming. Retries must not duplicate external actions.

Add the operational design

Use workload identities, restricted storage access, and Key Vault for unavoidable secrets. Define maximum run duration and tool-call limits. Scale workers against queue age and downstream capacity. Monitor completed-task rate, queue age, tail latency, dependency failures, and estimated cost. Keep tenant authorization in retrieval and cache keys. Release immutable artifacts and versioned agent configuration; test recovery, backup restoration, and incompatible data changes.

State your tradeoffs

I would start with managed container hosting unless Kubernetes control is required. I would use a queue for long work, accepting eventual completion in return for resilience and load smoothing. A second region adds cost and data complexity, so its need follows recovery objectives. Retrieval quality and model selection are evaluation decisions. I would explain the smallest useful architecture first, then show how it grows.

Self-check: Did you explain failure handling, tenant boundaries, idempotency, persistent state, quota, cost, monitoring, and release recovery? Avoid drawing every Azure service you know.

Production Troubleshooting Scenarios

41 Errors jump immediately after a release

I would stop further rollout, compare the new and old versions, and inspect user impact, traces, readiness, and recent configuration or schema changes. If the previous version is compatible with current data, I would roll back or route traffic back. I would verify recovery using successful user journeys and monitor delayed jobs. If rollback is unsafe because data changed, I would consider disabling the feature or applying a compatible forward fix. After restoration, I would reproduce the issue and improve the release checks. [S1]

Follow-up: What if the database migration removed a column? Explain why application rollback alone may fail.

42 Agent latency triples while CPU remains low

Low CPU does not rule out saturation. I would split end-to-end latency into queue wait, retrieval, database, model, and tool spans. I would compare prompt size, output length, retry count, connection pool wait, and model throttling before and after the change. If a dependency is limiting throughput, I would control concurrency or route only to a validated alternative. I would avoid simply increasing replicas, which may add pressure to the same bottleneck.

43 Queue age rises although workers are scaling

I would compare arrival rate with successful completion rate and inspect worker failures, processing duration, retry volume, and dead letters. More replicas cannot solve poison messages, database contention, or model quota saturation. I would isolate bad work, set retry limits, and check acknowledgement or lock handling. I would validate a concurrency setting that improves actual completions. During recovery, I would preserve tenant fairness and communicate expected delay rather than allow an uncontrolled drain that overwhelms downstream services.

44 A workload loses access to Key Vault

I would identify the error class first: authentication, authorization, DNS, networking, or service availability. I would check the workload identity binding, token audience, assigned permissions, resource scope, and recent changes. For private access, I would verify DNS resolution and network rules from the workload environment. A safe mitigation might restore a prior valid configuration. I would not make the vault public or grant broad administrator access to hide the issue. After recovery, I would add a check for the actual access path.

Answer order for any incident: impact -> recent change -> evidence -> safe mitigation -> verify recovery -> permanent improvement. Be ready to state what additional evidence would distinguish two plausible causes.

Behavioral and Recruiter Questions

45 How do you show ownership

I see ownership as following a change through deployment and ongoing operation. My CV includes release automation, production on-call work, RCA, and improvements to alerts and runbooks. I would choose a real example where I helped restore service and then completed prevention work rather than stopping when the alert cleared. I would explain what I personally did, how I involved the right people, and what the follow-up changed. The example should show reliable follow-through rather than imply I solved everything alone.

46 Tell me about disagreement with another engineer

Use a real situation rather than memorize a fictional conflict. A useful structure is: we had different views on a deployment or architecture choice; I clarified the constraints and failure risks; we compared a small experiment or production evidence; we documented the decision and responsibilities. Explain where you changed your own view if that happened. The outcome can be a better decision and working relationship, even when your original suggestion was not selected.

47 How do you balance deadline cost and reliability

I would separate essential service protections from improvements that can follow. Identity, recoverable releases, and visibility into production failures are difficult to defer safely. I would simplify the feature or architecture to meet the deadline, make the remaining risks explicit, and give the owner a clear choice with consequences. I would track temporary exceptions with owners and review dates. Cost decisions would preserve the agreed user outcome and service objective instead of reducing capacity without checking impact.

48 Why do you want this job and what are your logistics

I am interested in the combination of AI platform engineering and operational responsibility. It matches my backend, Azure, deployment, and production support experience while giving me room to deepen cost governance. For logistics, I would state my actual notice period, Abu Dhabi commute or relocation position, work authorization, and compensation expectations. I would not invent availability. I would also clarify how the permanent full-time employment description relates to the initial three-to-six-month project duration.

Questions to ask the interviewer

What is running today, and which reliability problem matters most? Who owns on-call, platform security, and cost approvals? What are the current model deployments, request volumes, and quotas? How are agent quality and successful tasks measured? What should this hire deliver in the first 90 days? How will employment and assignment continuation work after the initial project period?

Prepare one real mistake and learning story, one teamwork story, and one documentation example. Keep the focus on your actions and results, and avoid blaming previous colleagues.

Azure Compute and Networking

49 How do App Service Functions and Container Apps differ

I would compare the execution model first. App Service suits hosted web applications; Functions suits event-triggered code; Container Apps hosts containerized services and workers with managed operations. I would check runtime support, networking, scaling, execution duration, and operational requirements before selecting a service. A small representative workload is more useful than choosing from a feature list.

50 How do you handle cold starts

I would measure which part of initialization causes the delay, such as package loading, identity setup, or connections. I would reduce unnecessary startup work and consider warm capacity where the hosting plan supports it. The decision depends on latency requirements and idle cost. I would test after idle periods and deployments rather than assume steady-state measurements cover startup behavior.

51 How would you run a long background job

I would persist the job and place a durable dispatch request on a queue. Workers would checkpoint progress and use bounded retries, cancellation, and time limits. The web request would return a job identifier rather than remain open for minutes. I would verify whether work can be repeated safely and how a failed worker transfers ownership without duplicating external actions.

52 How would you select an Azure region

I would consider where users and data are located, permitted processing locations, service and model availability, recovery requirements, quota, and cost. I would measure latency to dependencies and verify the selected deployment types. I would also assess paired recovery resources and data movement. Geography alone does not guarantee every required service or model is available with sufficient capacity.

53 What is the role of a virtual network and subnets

A virtual network provides an address space and connectivity boundary for Azure resources. Subnets organize that space and support different workload and policy requirements. I would plan addresses to avoid overlap with connected networks, separate workloads where useful, and document routes and dependencies. Network segmentation complements identity controls; it does not replace authorization at the application or resource.

54 How do NSGs and a firewall differ

I would use network security groups for applicable network traffic rules around subnets or interfaces, and a firewall where centralized inspection and broader traffic policy are needed. I would evaluate direction, address, port, routing, and required application controls. Troubleshooting starts with the actual path: a correct rule is ineffective if traffic bypasses the intended inspection point.

55 How do you diagnose a private endpoint connection failure

I would check name resolution from the application environment, the resolved address, routing, security rules, endpoint approval, and service access configuration. I would separate connectivity from authentication and authorization errors. I would compare a working and failing client and examine recent changes. Making a service publicly reachable is not my default remedy for incorrect private DNS or routing.

56 What is the difference between Front Door and API Management

I would use Front Door for appropriate edge delivery and routing requirements, while API Management governs and exposes APIs through policies and contracts. They can be complementary. I would determine where authentication, throttling, caching, and routing belong and avoid duplicate or inconsistent policies. The exact topology follows the application, security requirements, and chosen service features.

Azure Identity and Storage

57 How would you apply least privilege

I would list the actions each identity needs, grant the narrowest useful permissions at an appropriate scope, and test denied operations. Deployment, runtime, and human support identities should have distinct responsibilities. I would review permissions periodically and after role changes. Least privilege must include data access and tool actions, not only the cloud management plane.

58 How do system assigned and user assigned managed identities differ

A system-assigned identity is associated with the lifecycle of its resource. A user-assigned identity is a separately managed resource that can be assigned where supported. I would choose based on lifecycle and governance needs, then restrict sharing to avoid excessive privilege. Neither choice eliminates the need to configure permissions or verify which identity the workload actually uses. [S23]

59 How would you provide emergency production access

I would agree on a controlled emergency process with restricted eligibility, time-limited access where supported, and an audit trail. The operator should know the incident scope and recovery steps. Afterward, I would remove temporary access and reconcile emergency changes into code. A standing broad administrator role for every engineer makes access easy but weakens accountability and containment.

60 What would you do after a credential leak

I would identify the affected identity and scope, revoke or rotate the credential, inspect usage, and contain unauthorized access. I would check copies in logs, artifacts, and deployment configuration, then deploy the replacement safely. I would verify dependent workloads and preserve evidence. Removing the secret from the latest commit alone does not invalidate it or remove previous exposure.

61 How would you handle large document uploads

I would avoid loading the whole upload into application memory. A controlled direct-to-storage flow can reduce API pressure, with scoped authorization, size limits, expiry, and validation. The service would persist metadata and process uploads asynchronously. I would handle incomplete uploads and verify file content before downstream parsing. User-supplied names should not determine unrestricted storage paths.

62 How would you protect uploaded documents

I would authorize tenant access, use restricted storage permissions, validate file type and size, and scan where required before parsing. I would define retention and deletion behavior, log access without exposing contents, and limit parser resources. Sensitive data needs protection in temporary files and derived indexes as well as the original object. The document processing worker should have narrow privileges.

63 How do you choose a retention policy

I would ask which data is needed for business use, audit, support, and recovery, then define retention by category. Documents, derived embeddings, agent state, backups, and telemetry can require different rules. I would document deletion behavior and validate it across copies. Indefinite retention increases cost and exposure; immediate deletion may conflict with agreed recovery or audit needs.

64 How would you recover an accidentally deleted object

I would first confirm the object, deletion time, and available protection settings. Recovery might use enabled soft delete, versioning, or a backup, depending on the service configuration. I would verify the restored content and permissions and assess downstream indexes. I would not promise recovery merely because the resource is hosted in Azure; recoverability depends on prior configuration and tested procedures.

Database Modeling and Transactions

65 How would you choose SQL or a document database

I would examine access patterns, relationships, transaction needs, consistency, query flexibility, and scale. A relational database is a strong starting point for structured run state and transactional records. A document store can suit appropriate document-oriented access patterns. I would model a representative query and failure case before choosing, and include indexing, operations, and cost in the comparison.

66 How would you model agent run state

I would store the run identifier, tenant, status, timestamps, configuration version, progress, cancellation state, and result references. Steps would record attempts and externally visible effects where necessary. I would define permitted state transitions and enforce them with concurrency controls. Large documents and outputs can be referenced rather than duplicated in every run record. Sensitive fields need explicit retention and access controls.

67 How do you choose a partition key

I would examine how queries distribute load and how data grows. A key should support common access patterns while avoiding a hot partition. Tenant ID may help isolation but can concentrate a very large tenant's traffic. I would consider composite strategies where supported and understand cross-partition query costs. The decision should use expected distributions, not only average tenant size.

68 What is eventual consistency and when is it acceptable

Eventual consistency allows copies or projections to converge over time instead of requiring every read to observe the latest change immediately. I would use it when the user workflow tolerates delay, such as a derived dashboard. I would make pending states visible and define maximum acceptable lag. Authorization and critical state transitions may require stronger guarantees than an analytics projection.

69 What is a lost update and how do you prevent it

A lost update occurs when concurrent writers overwrite each other's changes without detecting the conflict. I would use an appropriate transaction, row version or expected version check, or a suitable locking strategy. On conflict, I would retry only if the operation remains safe. For an agent run, two workers must not independently advance the same step and duplicate a side effect.

70 How do isolation levels affect correctness

Isolation controls what concurrent transactions can observe. Weaker isolation can allow anomalies; stronger isolation can require waiting or transaction retries. I would identify the invariant being protected and choose the database behavior that enforces it, then test competing transactions. I would not assume an isolation level has identical implementation details across database engines. [S12]

71 How would you investigate a database deadlock

I would capture the database's deadlock evidence and identify the transactions, resources, and order of access. I would reduce transaction duration, use consistent access order, and improve indexes where that reduces locking. A bounded retry may handle a victim transaction if repeating it is safe. Increasing timeout does not remove the circular dependency that caused the deadlock.

72 Why should a transaction not remain open during an LLM call

A slow or unpredictable external call can keep database locks and connections occupied, increasing contention and failure impact. I would persist a pending workflow state, perform the external call outside the transaction, then record its outcome with concurrency checks. Where side effects cross systems, I would use durable state and idempotency rather than pretend a database transaction covers the remote model or tool.

Database Performance and Caching

73 How would you diagnose a slow query

I would inspect the query plan, filters, joins, row counts, indexes, wait conditions, and recent data growth. I would measure representative parameters and distinguish database execution from connection-pool wait or network time. A suitable index or smaller result can help, but I would evaluate write overhead and storage. I would validate improvement using the same workload and tail latency.

74 What is the N plus one query problem

It occurs when one initial query triggers an additional query for each returned item. I would identify it through query counts and traces, then use an appropriate join, batch fetch, projection, or loading strategy. I would avoid fetching excessive related data just to reduce query count. The aim is less total work and predictable performance for the actual response shape.

75 How would you handle connection pool exhaustion

I would inspect active and waiting connections, long transactions, slow queries, leaked connections, and aggregate demand from all application replicas. I would ensure connections are released and impose bounded concurrency. Increasing the pool may shift overload to the database. I would measure whether the real limit is database capacity or poor application behavior before changing pool sizes.

76 When is read replication helpful

It can offload eligible reads and support recovery designs, but replicas can lag. I would direct only suitable queries to replicas and preserve required read-after-write behavior. I would monitor lag and define fallback behavior when replicas are unhealthy. Replication does not fix inefficient queries automatically and does not replace a tested backup against logical corruption or deletion.

77 Explain cache aside

The application checks the cache, loads missing data from the backing store, and populates the cache. I would set a deliberate expiry and handle updates so stale data stays within the allowed window. This improves suitable repeated reads but introduces consistency and invalidation concerns. Authorization-sensitive data must remain correctly scoped even when a cached value is reused. [S18]

78 What is a cache stampede

Many requests can miss or expire on the same key and hit the backing store at once. I would consider coalescing refreshes, staggered expiry, or controlled serving of permitted stale data. I would also limit fallback concurrency. The solution depends on freshness requirements; a financial or permission decision may not tolerate the same stale-data policy as a public reference list.

79 What belongs in a cache key for tenant data

The key must include the tenant and all relevant dimensions that change the authorized result, such as resource, version, locale, or access scope. I would avoid placing raw secrets or personal data in keys. I would review permission changes and invalidation. A cache hit should not bypass an authorization decision simply because the data was previously returned to another user.

80 What happens if Redis becomes unavailable

I would define which functions can safely fall back and which depend on Redis for correctness. Read caching may bypass to the database with strict load limits. A coordination or rate-limit function needs an explicit failure policy. I would monitor fallback impact and avoid turning a cache outage into a database outage. The policy should follow the consequences of allowing or rejecting work.

Messaging and Reliable Processing

81 How do queues and publish subscribe differ

A work queue commonly distributes tasks among competing consumers. Publish-subscribe sends an event to interested subscribers, each with its own processing responsibility. I would choose based on whether one worker should perform a job or multiple services should react independently. Delivery guarantees, ordering, retention, and retry behavior depend on the actual broker and configuration.

82 When would you consider Service Bus or Event Hubs

I would consider Service Bus for business messages and workflows that need broker features such as queues or subscriptions. Event Hubs suits high-volume event ingestion and stream processing patterns. I would verify ordering, retention, delivery, and consumer behavior for the chosen design. My decision would follow message semantics and processing requirements rather than throughput claims alone.

83 What is a consumer group

In stream-processing systems, a consumer group gives a logical consumer its own progress through the stream. Multiple groups can process the same events independently. Within a group, partition assignment affects parallelism. I would plan checkpointing and recovery and verify broker-specific behavior. Adding consumers beyond available partitions may not increase processing parallelism for that stream.

84 How would you version an event schema

I would prefer compatible additive changes and document the contract, field meanings, and defaults. Consumers should handle supported versions deliberately and ignore harmless extra fields where appropriate. Breaking changes need a transition plan and testing with old consumers. I would retain enough metadata for replay and distinguish the event's business time from its publication or processing time.

85 What belongs in a dead letter queue workflow

I would capture the reason for failure, alert the owner, and provide controlled inspection and reprocessing. Service Bus dead-letter messages require explicit handling; the queue is not a substitute for a recovery process. Before replay, I would correct the cause and assess idempotency. Sensitive payloads and diagnostic details need the same access protections as other production data. [S14]

86 How do you deal with duplicate messages

I would assume duplicates are possible and make processing idempotent. The consumer can use a durable event or operation identifier with a uniqueness constraint around its state change. I would define how long deduplication records remain useful and what happens on concurrent delivery. An in-memory set on one worker is insufficient when workers restart or messages reach another replica.

87 How would you preserve ordering

I would identify the smallest scope that requires order, such as one booking or one account, then choose an appropriate partitioning or session mechanism. I would record sequence or version information when needed and handle gaps deliberately. Global ordering can restrict throughput unnecessarily. I would also explain how retries or parallel consumers can affect the order observed by the application.

88 How do you safely replay historical events

I would select the scope and event versions, verify consumer compatibility, and isolate replay from live work where necessary. I would prevent duplicate external actions and control throughput. I would monitor progress and compare the resulting state with expected invariants. Replay can rebuild projections, but repeating irreversible side effects requires an explicit policy rather than ordinary consumer execution.

Microservices and API Contracts

89 What is a circuit breaker

A circuit breaker temporarily stops calls to a dependency after a configured failure pattern, then allows limited recovery checks. It differs from retrying each failed request. I would select thresholds and fallback behavior based on the dependency and user impact, and measure trips and recovery. It prevents repeated pressure on a failing service but cannot create a correct substitute result. [S15]

90 What is the bulkhead pattern

It isolates resource pools so failure or saturation in one workload does not consume all shared capacity. I might separate worker pools, concurrency budgets, or connection limits for critical and optional tasks. I would ensure the boundaries align with failure risks and monitor rejected work. Too many isolated pools can waste capacity, so the design needs a measured balance.

91 How do you propagate timeouts

I would establish an overall deadline and allocate smaller per-call timeouts so downstream work leaves time for cleanup and response. Cancellation should flow where supported, and retries must fit inside the remaining deadline. Independent long timeouts at every layer can make total latency unpredictable. I would monitor timeout location to distinguish overload from genuine dependency failure.

92 When would you use a saga

A saga coordinates a business workflow across services through local transactions and compensating actions. I would use it when one database transaction cannot cover the workflow. I would record progress and retries durably and define failure handling at every step. A compensation is a business operation, not always a perfect reversal; sending an email or exposing a report may be irreversible.

93 What makes an API change backward compatible

Existing clients should continue to operate with the agreed meaning of fields, status codes, and behavior. Adding an optional field can be compatible; changing a type or making an optional field required can break clients. I would use contract tests and staged adoption. Compatibility includes timing and errors, not only whether a JSON document still parses.

94 How would you design pagination

I would choose a stable ordering and bound page size. Cursor pagination often suits changing datasets and large offsets, while offset pagination can be adequate for simpler cases. I would define filter and cursor validity, avoid exposing internal secrets, and test inserts or deletes between requests. Authorization must apply to each page rather than only the initial request.

95 How would you design API error responses

I would return consistent machine-readable codes, useful safe messages, and a correlation identifier. Validation errors should identify the relevant input without disclosing sensitive data. HTTP status should reflect the failure class, and clients should know which errors are retryable. I would retain detailed internal diagnostics in controlled telemetry rather than reveal stack traces or credentials in the response.

96 When would you choose REST GraphQL or gRPC

I would use REST for straightforward resource-oriented integration, GraphQL when clients need flexible composition with controlled query costs, and gRPC for suitable typed service-to-service calls. I would consider client support, observability, caching, security, and operational complexity. A mixed architecture can be reasonable, but each interface needs a clear purpose and ownership.

API Security and Python

97 What should you validate in a JWT

I would validate the signature against trusted keys, expected issuer and audience, lifetime, and required claims. Then I would separately authorize the requested action and resource. I would handle key rotation through the identity provider's supported mechanism. Decoding the payload alone does not authenticate it, and accepting any valid token without checking audience can allow a token intended for another service.

98 How do you choose a rate limiting policy

I would identify what is scarce or expensive and limit by an appropriate authenticated identity, tenant, or operation. Request count alone may not reflect model token or compute cost. I would provide clear overload responses and retry guidance, protect critical traffic, and monitor fairness. Distributed enforcement needs a shared or otherwise coordinated view of allowances where the policy requires it.

99 Does CORS secure an API

CORS controls which browser origins can read permitted cross-origin responses; it is not an authentication or authorization mechanism. I would still validate identity, permissions, and request semantics. Nonbrowser clients are not constrained in the same way. If cookies authenticate requests, I would separately consider CSRF protections. A permissive CORS setting should not become a substitute for a security design.

100 How would you protect against SSRF through an agent tool

I would avoid arbitrary user-controlled destinations where possible and restrict schemes, hosts, ports, and reachable networks. I would validate resolved addresses and redirects and enforce outbound network controls. The tool would have limited credentials and time or size limits. A string check on the original URL alone is insufficient because resolution and redirects can change where the request goes. [S19]

101 What are generators useful for

Generators produce values incrementally, which can reduce memory use for large or streamed datasets. I would use them when downstream work can also process incrementally. A later conversion to a list removes much of the memory advantage. I would consider cleanup, exceptions, and whether iteration can be repeated, especially when a generator reads a file or network stream.

102 Why are mutable default arguments dangerous

A mutable default object can be shared across calls, creating surprising retained state. I would normally default to None and create a fresh list or dictionary inside the function when needed. I would also inspect shared class attributes and caches for similar unintended state. A small repeated-call test helps reveal this issue before it becomes a request-isolation bug.

103 When would you use threads or processes in Python

I would use an approach that matches the work and interpreter environment. Threads can help with blocking I/O; processes can provide parallel execution and isolation for CPU-heavy tasks. Async clients suit cooperative I/O. I would account for serialization, process startup, library behavior, and deployment limits. I would benchmark the actual workload rather than assume one concurrency model is always faster.

104 How would you manage Python resources safely

I would use context managers and try/finally where needed to close files, transactions, and connections even on failure. For asynchronous resources, I would use their async lifecycle mechanisms. I would keep ownership explicit and avoid opening a client for every tiny operation unnecessarily. Cancellation and exceptions should not leave a connection or temporary file permanently occupied.

FastAPI Node js and TypeScript

105 How do you decide between def and async def in FastAPI

I would use async def when calling suitable awaitable clients, and understand how the framework handles ordinary synchronous path functions. A blocking call directly inside an async endpoint can block its event loop. I would inspect the libraries used and move unsuitable work to an appropriate execution context. Merely changing the function declaration does not make synchronous I/O nonblocking. [S9]

106 What would you initialize at service startup

I would initialize reusable clients, validate required configuration, and establish resources whose lifecycle matches the application. Startup should fail clearly for unrecoverable configuration errors. I would avoid performing lengthy migrations independently in every replica. Shutdown should release clients and finish or safely stop active work. Resource initialization belongs in an explicit lifecycle path with tests for startup failure.

107 Are in process background tasks durable

Generally, they depend on the application process staying alive and may be unsuitable for work that must survive restarts. I would use a durable queue and worker for critical or long-running jobs. I would define accepted-work semantics, persistence, retries, and status tracking. Lightweight post-response work can be appropriate when its loss is tolerable and the framework behavior is understood.

108 How would you validate an API request in Python

I would use an explicit request schema, constrain lengths and sizes, and validate business rules after structural validation. I would reject unknown or conflicting input according to the contract and avoid trusting client-supplied authorization fields. I would keep response schemas separate where sensitive internal fields must be excluded. Type annotations help communicate intent but do not enforce every business invariant.

109 Why can a Node service be slow with few active requests

A callback can monopolize the event loop through heavy computation, large synchronous operations, or expensive input processing. I would measure event-loop delay and profile the workload, then partition work or move suitable work to workers. I would also inspect the worker pool and downstream waits. Low request volume does not mean each request performs little work. [S10]

110 What does TypeScript fail to validate at runtime

Static types do not validate arbitrary JSON, environment variables, database records, or external messages at runtime. I would use explicit boundary validation and then work with trusted types internally. An assertion or cast does not transform the data into the declared shape. I would test malformed payloads and avoid allowing unchecked external input to reach privileged tool calls.

111 Why can Promise all cause overload

It can start a large number of operations together without limiting concurrency. I would use bounded batches or a worker pool when the dependency has capacity or quota limits. I would define cancellation and partial-failure behavior and preserve relevant results. Parallelism reduces time only while the bottleneck has spare capacity; uncontrolled fan-out can increase latency and error rates.

112 How would you handle asynchronous errors in Node

I would await or deliberately manage each promise, map expected failures to consistent responses, and ensure background tasks have explicit error handling. Unexpected process-level failures need a defined shutdown and restart strategy. I would include context in logs without leaking secrets. Catching every error and returning success hides failed work and makes diagnosis more difficult.

ASP NET Core and Frontend Integration

113 How do dependency injection lifetimes affect a service

Transient, scoped, and singleton services have different reuse and ownership behavior. I would choose based on state, thread safety, and resource lifecycle. A singleton should not capture a request-scoped dependency improperly. For background work, I would create an appropriate scope rather than reuse a completed request's resources. A lifetime mistake can cause leaked state, concurrency faults, or disposed-object errors.

114 Why avoid blocking on async work in ASP NET Core

Blocking waits consume request-processing capacity and can contribute to thread-pool starvation under load. I would keep I/O paths asynchronous end to end where supported and propagate cancellation. I would measure throughput and tail latency rather than add threads as the first fix. CPU-heavy work needs a separate capacity decision; async does not make the computation cheaper. [S11]

115 How would you use HttpClient safely

I would use a supported reuse and configuration pattern, such as a suitable factory or managed client lifetime, rather than casually create and dispose a new client for every request. I would configure deadlines, connection behavior, and dependency policies. I would avoid layering retries without understanding their combined effect, and verify behavior during DNS changes and dependency outages.

116 How do you decide between EF Core and direct SQL

I would consider maintainability, query complexity, performance, and transaction requirements. EF Core can be productive for common data access; direct SQL can be suitable for carefully optimized or specialized queries. I would inspect generated SQL, avoid excessive loading, and use parameters. The decision should follow measured requirements rather than a claim that an ORM is inherently too slow.

117 How would you stream agent progress to a frontend

I would define events and terminal states, choose a suitable transport such as SSE or WebSockets, and preserve authorization and reconnect behavior. The UI should distinguish partial output from a completed result. I would decide whether events can be replayed and how the client detects gaps. Streaming improves perceived progress but does not replace durable run state.

118 What should the UI show for a long running task

I would show the run state, meaningful progress where available, cancellation options, and a stable link or identifier. After refresh, the UI should restore status from the server. Failure should provide a useful next action rather than silently restart the task. The design must not imply success merely because the server accepted the request or produced partial output.

119 How would you handle frontend API contract changes

I would agree on the contract with frontend engineers, provide realistic examples, and test both error and success flows. New fields or endpoints would be adopted in stages while older clients remain supported for the agreed window. I would use contract validation where useful. Shared TypeScript types are helpful but do not guarantee compatibility with a separately deployed backend.

120 How do you keep a frontend from exposing credentials

I would keep privileged service credentials on controlled backend services and use the user's appropriate authentication flow for browser access. Build variables shipped to a browser should be considered public. I would review logs, source maps, and client telemetry for accidental exposure. If direct storage access is needed, I would issue narrowly scoped short-lived access through an authorized backend.

CI Pipelines and Releases

121 What should be cached in a build pipeline

I would cache reproducible dependencies or intermediate work with keys that reflect relevant lockfiles, toolchains, and build inputs. I would avoid caching credentials or mixing untrusted and privileged artifacts. A cache miss should remain correct. I would measure whether caching saves meaningful time and ensure stale entries cannot silently change the released artifact or bypass a required verification step.

122 How would you handle a flaky test

I would capture conditions and determine whether the flakiness exposes a real race, uncontrolled dependency, or test problem. I would fix isolation, synchronization, or the underlying defect. Temporary quarantine needs an owner and deadline. Blind retries can hide genuine failures. I would track flake rate and avoid weakening important release checks solely to make the dashboard green.

123 How do you secure pull request pipelines

I would treat code from untrusted changes as untrusted execution. I would restrict secrets, token permissions, privileged runners, and artifact promotion paths. Production deployment should require the intended reviewed context. I would pin or review external actions as appropriate and examine how scripts use input. A successful test run should not automatically grant arbitrary production access.

124 How do you prove what was deployed

I would record the source revision, immutable artifact digest, build run, configuration version, and deployment history. Release metadata should appear in operational telemetry and a deployment record. I would verify that promotion uses the same artifact rather than a new build. This makes incident comparison and recovery more reliable than relying on a mutable tag called latest.

125 How do blue green and canary deployments differ

Blue-green maintains separate versions and changes traffic between them; canary exposes a limited portion of traffic before expanding. I would choose based on routing support, cost, state, and the signals available to judge health. Both require a plan for data compatibility and recovery. A second environment alone does not guarantee that switching back will reverse every side effect.

126 What metrics should stop a rollout

I would use signals tied to user outcomes, such as elevated error rate, latency, failed completions, or unexpected tool behavior. I would compare the new version with a relevant baseline and require enough observation to make the decision useful. Thresholds and sample sizes depend on traffic and risk. I would also watch dependency pressure and cost anomalies rather than only container readiness.

127 How would you remove an obsolete database field

I would first deploy code that tolerates both old and new schema and migrate data if necessary. After verifying consumers no longer depend on the old field, I would remove it in a later controlled change. I would include background jobs and reports in dependency checks. Deleting the field in the same release as the first code change makes rollback harder.

128 When are feature flags useful

They can separate deployment from exposure and provide a targeted way to disable a feature. I would define safe defaults, ownership, authorization for changes, and removal dates. Flags must be tested in meaningful combinations and observed in telemetry. A flag cannot reverse irreversible data changes automatically, and accumulating abandoned flags makes behavior harder to reason about.

Terraform Bicep and Governance

129 Why protect Terraform state

State maps configuration to managed resources and can include sensitive values. I would store it in an appropriately protected remote backend, restrict access, enable suitable recovery protections, and use locking where supported. I would separate unrelated failure domains deliberately. Marking a value sensitive can reduce display exposure, but it does not mean no sensitive value exists in the state. [S22]

130 How would you organize Terraform modules

I would create modules around coherent infrastructure responsibilities with clear inputs and outputs, keeping environment configuration separate. I would avoid abstractions that hide important lifecycle or security behavior. Module and provider changes would be versioned and reviewed. I would start with repeatable patterns actually needed by the project rather than make every single resource a separate generic module.

131 What would you review in a Terraform plan

I would inspect creates, replacements, and deletions; permission and network changes; state or address changes; and impacts on data-bearing resources. I would confirm the environment and input values and ensure the plan corresponds to the reviewed code. An apparently small configuration edit can trigger a disruptive replacement, so I would assess operational consequences before applying it.

132 How would you bring an existing resource under IaC

I would inventory its configuration and dependencies, write the matching configuration, and use the tool's appropriate import mechanism. I would review the resulting plan until it reflects intended changes rather than accidental replacement. I would back up relevant state and document ownership. Importing an identifier does not automatically produce a correct long-term configuration or settle drift decisions.

133 What does Bicep what if tell you

It previews expected resource changes before deployment and is useful for review, but it has limitations. I would inspect changes alongside dependency behavior and operational impact and verify the target scope and parameters. I would not treat a preview as proof that the application remains healthy or that no data will be affected. A post-deployment validation remains necessary. [S2]

134 What is Azure Policy for

I would use Azure Policy for applicable resource governance, such as evaluating or enforcing agreed configuration requirements. I would test policies and their effects before broad enforcement and assign exception owners. Policy complements access control and application security. A compliant resource configuration does not guarantee the workload's business logic or tenant authorization is correct.

135 How would you standardize resource tags

I would define a small required set, such as owner, environment, application, and cost allocation category, then apply it through infrastructure modules and suitable governance. I would validate coverage and keep ownership current. Tag-based reporting helps only if tags reflect reality. I would also understand which costs cannot be cleanly allocated through resource tags alone.

136 How do locks differ from permissions

Resource locks can add protection against certain management actions, while permissions determine which actions an identity can perform. I would use each deliberately and verify the affected operation scope. Locks should not be treated as a backup or a comprehensive defense against data changes. Operational procedures must account for them so legitimate recovery does not stall unexpectedly.

Docker and Kubernetes Networking

137 Why use multi stage builds

They separate build tooling from the final runtime image, helping reduce unnecessary contents. I would copy only required output and verify runtime dependencies. Smaller images can improve transfer and reduce exposure, but size is not the sole security measure. I would scan, patch, and configure the final image and validate that diagnostic and startup requirements are still met. [S21]

138 How would you design graceful shutdown

The process would stop accepting new work, signal active tasks, and finish or checkpoint them within an agreed grace period. Workers should release or settle messages appropriately and avoid claiming success for incomplete work. I would test termination during requests and jobs. A graceful path must fit the platform's termination behavior rather than assume unlimited time before the process is killed.

139 Why can a container work locally but fail in production

I would compare architecture, environment configuration, filesystem permissions, identity, network access, resource limits, and entrypoint behavior. Local development may provide files or credentials that are absent in the image. I would reproduce with the same artifact and relevant runtime restrictions. Rebuilding a different image locally can conceal the exact production issue instead of explaining it.

140 How do you reduce container supply chain risk

I would use trusted sources, track dependencies, scan artifacts, record provenance, and deploy immutable references. I would keep build credentials out of images and review third-party build steps. Image signing helps establish origin when verification is enforced. It does not prove that the signed code is vulnerability-free or safe for every runtime permission.

141 How do a Service and an Ingress differ

A Service provides a stable way to reach selected workloads inside the cluster or through supported exposure types. Ingress describes applicable HTTP routing handled by a controller. I would inspect selectors, endpoints, ports, routing rules, and TLS when diagnosing connectivity. The controller and underlying networking matter; defining an Ingress object alone does not provide a working traffic path.

142 How would you troubleshoot service discovery

I would test resolution and connectivity from the affected workload, inspect service names, namespaces, endpoints, and network policy, and compare with healthy clients. I would distinguish DNS failure from a service with no ready backends. I would also inspect recent deployment and cluster changes. Testing only from my laptop does not validate the workload's in-cluster path.

143 How do ConfigMaps and Secrets differ operationally

They hold different kinds of configuration, but storing a value in a Kubernetes Secret is not by itself a complete security design. I would restrict access, protect storage, avoid exposure in logs, and manage rotation. I would check how applications observe changes and whether restart is required. External secret systems may be appropriate when lifecycle and governance require them.

144 What should a network policy achieve

I would use supported network policy enforcement to limit communication to required paths and reduce lateral movement. I would begin with a clear dependency map and test DNS, telemetry, identity, and application traffic before rollout. Policies need a functioning enforcement implementation. A policy file that is accepted by the API is not sufficient evidence that the expected traffic is blocked.

Kubernetes Capacity and Lifecycle

145 What are resource requests and limits

Requests inform scheduling and resource allocation decisions; limits constrain resource use according to the resource type and runtime behavior. I would set them from measured workload needs and monitor throttling or memory termination. Overstated requests waste capacity; understated requests can harm placement and performance. I would account for startup peaks and workload variability rather than choose the same values for every service. [S20]

146 Why might pods remain Pending

I would inspect scheduler events for insufficient resources, affinity constraints, taints, volume issues, or other placement requirements. I would check whether autoscaling can add the needed node type and whether quotas or limits prevent it. Increasing replicas does not resolve unsatisfiable scheduling constraints. I would identify the missing requirement and apply the smallest appropriate correction.

147 What are taints tolerations and affinity used for

They influence placement by discouraging or allowing scheduling on certain nodes and expressing placement preferences or requirements. I would use them to separate workloads or meet operational needs without creating impossible constraints. I would check their interaction with available capacity and autoscaling. A toleration permits placement on a tainted node; it does not guarantee that the pod will run there.

148 How would you protect one tenant from another noisy workload

I would define concurrency and resource boundaries, prioritize critical work, and consider separate queues or worker pools. In Kubernetes, namespace and scheduling controls can contribute, but application-level fairness is still needed. I would monitor tail latency and rejected work by tenant. Maximum replicas alone cannot prevent a single tenant from consuming every downstream model or database allowance.

149 What is a PodDisruptionBudget for

It constrains certain voluntary disruptions to preserve a desired level of availability. I would align it with replica count, placement, and maintenance procedures. It does not prevent all failures or guarantee application health. A configuration that permits no useful maintenance can stall node updates, so I would test both service continuity and the ability to perform planned operations.

150 How would you upgrade a production cluster

I would verify supported version paths, workload API compatibility, add-ons, capacity, and maintenance constraints. I would test in a representative environment, plan node upgrades and disruptions, and monitor service behavior during rollout. Recovery options depend on the platform and change type, so I would not promise a simple in-place downgrade. Application recovery and infrastructure recovery need separate plans.

151 What does GitOps reconciliation do

A controller compares observed resources with declared desired state and attempts to reconcile differences according to its configuration. I would review permissions, sync behavior, drift handling, and rollout health. It improves repeatability and history but can also undo manual emergency changes. I would coordinate emergency fixes with the declared state and record why any reconciliation suspension was needed.

152 What happens when a rollout never becomes healthy

I would inspect the Deployment conditions, new pod failures, readiness, events, and available old replicas. I would stop promotion and take the appropriate recovery action. Kubernetes reports rollout problems, but a Deployment should not be assumed to automatically choose and execute the desired rollback policy. The delivery system must define how failed health checks lead to recovery. [S17]

Telemetry KQL and Dashboards

153 How do metrics logs and traces complement each other

Metrics summarize behavior, logs describe events, and traces connect operations across a request or workflow. I would use metrics to detect a problem and traces or logs to investigate its path and cause. Common identifiers help correlation. I would deliberately choose what to collect, because more telemetry can increase cost and exposure without improving diagnosis. [S13]

154 What is high cardinality and why does it matter

A metric label with many distinct values can create a large number of series and increase storage or processing cost. I would avoid using per-run IDs or arbitrary user input as ordinary metric labels. Those identifiers may be better suited to controlled logs or traces. I would keep useful low-cardinality dimensions and verify the actual monitoring backend's behavior and limits.

155 How would you choose a trace sampling strategy

I would consider traffic, cost, important workflows, errors, and diagnostic needs. Sampling should preserve enough evidence for useful investigations and remain coherent across services. I would understand whether the system decides before or after a trace completes and what the chosen tools support. Sampling cannot be used to calculate every business total without accounting for what was omitted.

156 How do you instrument an asynchronous workflow

I would carry appropriate correlation context from request to job and record producer and consumer operations. The job should have a durable run identifier independent of one process. I would model retries and separate processing attempts clearly. I would avoid inventing one continuous synchronous span for work that is delayed or resumed; the tracing representation should reflect the actual workflow.

157 How would you find a rise in errors using KQL

I would select the relevant log table, filter the time range and service, group failures into useful time bins, and compare with request volume. I would then split by version or dependency and inspect correlated traces. Table and field names depend on the telemetry schema. I would validate the query against real records instead of assuming every workspace has the same columns. [S16]

158 What makes a useful service dashboard

I would show user-facing health, workload volume, latency distribution, failed completions, dependency pressure, and deployment changes. Each chart should answer a clear operational question and link to investigation context. I would separate executive trends from on-call diagnostics. A dashboard full of CPU and memory charts can miss failed business operations or a broken model integration.

159 How would you alert on missing telemetry

I would define expected reporting behavior and compare it with received data, accounting for idle periods and ingestion delay. I would use a heartbeat or independent synthetic check where appropriate. The absence of errors is not proof of health when the service cannot report. I would test what happens if the collector or monitoring path fails, not only the application.

160 How do you avoid logging sensitive prompts

I would collect minimal metadata by default, such as run ID, model configuration, token counts, durations, and classified errors. Any content logging would require a deliberate purpose, access controls, redaction, and retention. I would inspect SDK and debug logging behavior. Removing content from one application log does not guarantee that intermediaries or external tools are not recording it.

SRE and Recovery

161 How would you choose an initial SLO

I would identify the user journey, define good and total events, choose a time window, and agree on the business target. I would check current performance and dependency constraints before promising a number. I would include correctness or completed-task behavior where appropriate for AI workflows. A target that cannot be measured consistently will not guide useful reliability decisions.

162 What is error budget burn rate

It describes how quickly the service is consuming its allowed unreliability relative to the budget for the chosen window. I would use it to identify sustained or fast degradation and guide response or release risk. I would define eligible events and alert windows carefully. One short error spike and a long mild degradation can require different operational actions.

163 How do you prioritize reliability work

I would rank issues by user impact, recurrence, exposure, and the value of the proposed improvement. I would include recovery gaps and the cost of operational toil. I would validate that the fix addresses the observed failure rather than only silence an alert. Clear owners and measurable completion criteria make the backlog useful alongside feature delivery.

164 What should a production runbook contain

It should identify the symptom, likely evidence sources, safe diagnostic steps, mitigation choices, required permissions, escalation path, and recovery checks. Commands need context and clear scope. I would keep it tested and linked from alerts. A runbook should help a tired engineer make a safe decision rather than prescribe a destructive action without explaining when it applies.

165 Why is replication not the same as backup

Replication can copy an erroneous update or deletion quickly, while a suitable backup preserves a recoverable prior state. I would design protection against both infrastructure failure and logical mistakes. I would check retention, isolation, permissions, and restore procedures. A healthy replica does not prove that yesterday's corrupted data can be recovered to the required point.

166 How do you test a backup

I would restore into an isolated environment, validate integrity and application behavior, measure restore time, and compare recovered data with the required recovery point. I would test access to keys and dependencies needed for recovery. A successful backup job is only one signal. The useful proof is that the team can restore a working service within the agreed constraints.

167 How would you plan regional failover

I would map compute, data, networking, identities, secrets, model availability, and routing in the recovery region. I would define activation, data reconciliation, and return procedures and test the process. I would verify capacity and quota rather than assume they are available during an incident. Regional recovery can be limited by the least-prepared dependency, even when application containers are portable.

168 What should happen after recovery

I would verify user-facing behavior, delayed work, data integrity, and the stability of dependencies. I would remove temporary mitigations only when safe and communicate residual risks. I would preserve a timeline and assign prevention actions. For disaster recovery, I would also plan returning to the normal topology; an untested return path can cause another outage.

Infrastructure and LLM Costs

169 How do you identify idle resources

I would combine inventory, ownership, usage metrics, and cost reports. Low CPU alone does not prove a resource is unnecessary; it may provide redundancy or store critical data. I would confirm purpose with the owner and define a reversible shutdown or removal plan where appropriate. I would also check orphaned disks, test environments, and resources no longer referenced by deployments.

170 When would you make a capacity commitment

I would first right-size and understand the stable baseline and applicable service terms. A commitment can suit predictable eligible usage, but variable or temporary workloads may not justify it. I would compare utilization assumptions and organizational plans with the financial owner. I would avoid treating a discounted rate as savings when the committed capacity is not actually used.

171 How do logs affect cloud spending

Ingestion, storage, retention, and query behavior can contribute to telemetry cost. I would remove unnecessary verbose events, choose appropriate retention by purpose, and use metrics or sampling where suitable. I would preserve critical audit and incident evidence. Reducing cost by disabling all logs can make outages harder to recover from and create a larger operational cost.

172 How would you measure cost efficiency

I would use a unit tied to the business outcome, such as cost per successfully completed agent task, alongside total spend and quality. I would allocate shared infrastructure transparently and record assumptions. Comparing raw monthly spend can be misleading when traffic changes. Efficiency gains need to preserve the agreed service behavior and be measured over a representative workload.

173 What would a per tenant usage ledger record

I would record authorized tenant, run and call identifiers, model deployment, token usage where available, estimated charges, timestamps, and configuration version. I would reconcile estimates with billing and handle missing or delayed usage. Concurrency controls are needed when multiple calls consume an allowance simultaneously. Sensitive prompts should not be required to measure ordinary usage and cost.

174 How would you enforce a spending allowance before a model call

I would estimate and reserve a bounded allowance atomically, then settle it against reported usage after the call. I would define what happens on timeout or missing usage and retain a conservative reservation until reconciled. The estimate and billing model must match the deployment. Checking a balance without reserving capacity can let parallel calls exceed the same allowance.

175 When is response caching safe for AI

I would require matching authorization context, input meaning, relevant configuration, and acceptable freshness. I would include tenant and permission dimensions and avoid reuse across sensitive boundaries. Nondeterministic outputs or changing underlying documents can make caching inappropriate. I would measure hit quality and invalidation behavior rather than assume similar text always deserves the same cached answer.

176 How do you avoid retry driven cost growth

I would classify failures, bound attempts, honor deadlines, and account for retries at every layer. I would prevent repeated successful side effects and inspect whether partial responses or unknown outcomes should be retried. I would track attempts and cost per completed task. Retrying every failure indefinitely can increase spend while making dependency overload worse.

RAG and AI Evaluation

177 How would you choose document chunk size

I would test representative documents and queries, considering structure, retrieval precision, context limits, and duplicated overlap. Very small chunks can lose meaning; very large chunks can dilute useful evidence and increase context cost. I would evaluate answer support and retrieval quality rather than choose a universal size. Metadata and stable references help explain where evidence came from.

178 What is hybrid retrieval useful for

It combines different retrieval signals, commonly lexical matching and vector similarity, to improve relevant evidence for varied queries. I would test it against the actual corpus and apply authorization filters correctly. Ranking and reranking need latency and quality evaluation. More retrieval components add operational complexity, so I would retain them only when the measured benefit justifies the cost.

179 How would you keep an index fresh

I would track source changes and ingestion state, process updates and deletes reliably, and record index versions or timestamps. I would monitor lag and failed documents and provide controlled reprocessing. Permission changes must propagate as well as content changes. A system that refreshes text but retains obsolete access metadata can return evidence to users who no longer have permission.

180 How would you investigate poor retrieval

I would examine whether the relevant source exists, was parsed correctly, is indexed, passes authorized filters, and appears in candidate results. I would compare query formulation, embedding or ranking behavior, and chunk structure. I would separate missing evidence from generation errors. Adjusting the prompt cannot fix a document that was never ingested or is incorrectly excluded.

181 How would you build an evaluation dataset

I would gather representative tasks, expected evidence or outcomes, and important failure cases, with appropriate handling of sensitive data. I would include easy and difficult examples and keep a held-out set for comparison. I would version the dataset and document scoring criteria. A few handpicked successful prompts are not sufficient evidence that a change improves the production workload.

182 What would you measure besides answer accuracy

I would measure supported evidence, task completion, authorization behavior, harmful tool actions, latency, cost, and stability across repeated runs where relevant. I would separate retrieval and generation quality. Metrics should reflect the user workflow rather than only linguistic style. I would also inspect important failures manually because one aggregate score can hide a severe boundary violation.

183 What are the limitations of using an LLM as a judge

It can help scale evaluation but may have bias, inconsistency, or blind spots. I would calibrate it against human-reviewed cases, use explicit criteria, and inspect disagreements. I would avoid relying on one judge for critical authorization or safety claims. Version changes in the judge can alter scores, so evaluation configuration needs the same discipline as application configuration.

184 How would you detect quality drift

I would monitor a stable evaluation set and representative production outcomes, segmented by task and configuration. I would inspect changes in documents, traffic, model deployment, prompts, and tools. I would define an escalation and recovery path for unacceptable degradation. User feedback is valuable, but its absence does not prove the platform is maintaining quality.

Agent State and Advanced Incidents

185 When would you use several agents instead of one

I would use multiple agents when distinct responsibilities or controlled parallel work produce a measurable benefit. I would first compare a simpler single-agent or fixed workflow design. Multiple agents require coordination, durable state, permission boundaries, and additional debugging. I would measure task success, latency, and cost rather than assume a larger agent team is inherently more capable.

186 How would you checkpoint an agent run

I would persist completed steps, relevant state, configuration versions, and known external effects at deliberate boundaries. On resume, I would verify which steps are safe to repeat and whether permissions or inputs changed. The checkpoint needs enough information to recover without leaking unnecessary data. Storing only the conversation text may omit tool outcomes and irreversible business actions.

187 What if cancellation arrives while a tool is running

I would record cancellation durably and stop scheduling new work. I would attempt cancellation of the active call if supported, but acknowledge that the tool may already have committed an action. I would reconcile the final outcome and report an accurate state. Cancellation is not automatically rollback, so the user should know whether any effects remain.

188 How would you add a human approval step

I would persist a pending approval state with a clear action description, authorized approver, expiry, and audit record. Approval must bind to the exact intended action and relevant inputs; changed inputs may require new approval. The workflow should handle rejection, timeout, and duplicate responses. A message saying approved is not sufficient without verifying identity and matching the action.

189 What would you do if production DNS fails

I would determine scope and test name resolution from affected workloads, then inspect resolver configuration, private zones, records, and recent network changes. I would compare direct connectivity carefully without bypassing TLS or authorization. A temporary fix should preserve security and be reversible. I would monitor recovery and cache behavior and reconcile the corrected configuration into the normal deployment path.

190 What if a certificate expires during an incident

I would identify the failing endpoint and certificate chain and restore a valid certificate through the supported rotation process. I would verify propagation and client behavior and avoid disabling verification as the normal fix. I would check why renewal or alerting failed and add tested renewal monitoring. The incident can affect different layers, including ingress, outbound dependencies, or mutual TLS.

191 What if the database is healthy but the API times out

I would investigate connection-pool wait, network paths, application thread or event-loop saturation, transaction contention, and per-call deadlines. Healthy aggregate database metrics may hide one problematic query or tenant. I would inspect the request's trace and compare working paths. I would choose a targeted mitigation rather than assume the database needs more capacity.

192 What if one agent tool repeatedly performs the same action

I would pause or disable the affected path, inspect run state and attempt history, and check idempotency, timeout handling, and checkpoint recovery. I would identify actions already committed before allowing replay. A tool that succeeds remotely but times out locally can trigger duplicates. Prevention needs durable operation identity and reconciliation, not only a prompt telling the agent not to repeat itself.

Coding Practice and Leadership

193 How would you design a bounded worker pool

I would cap active work, queue size, and wait time, with defined rejection or backpressure behavior. Each task would have a deadline, cancellation handling, and observable outcome. I would ensure failures release permits and resources. I would test overload and shutdown. A semaphore limits concurrency, but a separate policy is needed to prevent unlimited waiting tasks from accumulating.

194 How would you test an idempotent endpoint

I would send the same key and payload repeatedly and concurrently, then verify one logical operation. I would test conflicting payloads, crash boundaries, in-progress results, expiration, and cross-tenant keys. I would inspect durable records and external side effects, not only response status. A test that calls the endpoint twice sequentially may miss the race that matters in production.

195 How would you design a distributed rate limiter

I would choose a rate model, identity scope, time behavior, and consistency requirement, then use an atomic shared mechanism or a deliberately bounded distributed approximation. I would define failure behavior and account for clock and network effects. I would test concurrent callers across replicas. A separate in-memory counter on each replica cannot enforce one exact global allowance.

196 How would you debug a memory leak

I would compare memory over time with request or job volume, distinguish heap from other allocations, and capture suitable profiles or dumps safely. I would look for retained objects, caches, buffers, listeners, or unreleased resources. I would reproduce the workload and verify that the fix stabilizes memory. Increasing limits can buy time but does not remove the retained allocation.

197 How would you hand over an unfamiliar service

I would provide an architecture and dependency map, access and ownership information, deployment and recovery procedures, dashboards, known risks, and the current backlog. I would walk through one deployment and one incident scenario with the receiving engineer. A handover is complete when they can operate the service, not merely when I have sent a long document.

198 How do you communicate risk to a nontechnical stakeholder

I would explain the affected user outcome, likelihood or uncertainty, practical consequence, and available options in plain language. I would state what we know and what additional evidence would change the decision. I would avoid presenting an implementation detail as the decision itself. Clear ownership and next steps help the stakeholder choose without needing to diagnose the system.

199 How would you mentor an engineer during an incident

I would assign a clear bounded investigation task, explain the evidence needed, and keep risky changes coordinated. I would preserve space for learning without delaying recovery or giving unreviewed broad production access. Afterward, I would review the reasoning and update the runbook together. The goal is stronger judgment and confidence, not just memorizing a sequence of commands.

200 What would you do if asked to promise an unrealistic deadline

I would clarify the required outcome and identify the work and constraints that drive the estimate. I would propose a smaller deliverable or phased approach and make reliability and security consequences explicit. I would communicate uncertainty early and update the estimate as evidence changes. I would commit to a credible scope rather than promise a date that depends on hidden assumptions.

Practical Exercises with Answer Checklists

These are design exercises for practice. Implement them locally if time allows; the outlines below are not complete production code.

Exercise 1 Implement a run API in Python or TypeScript

Build POST /runs and GET /runs/{id}. Accept an idempotency key, return 202 for accepted work, and persist state. Answer checklist: validated schema; tenant from verified identity; authorization on status reads; unique tenant/key constraint; request-hash conflict handling; durable dispatch intent; consistent error responses. Test a duplicate request, a conflicting request, cross-tenant access, and a crash between transaction commit and publication.

Exercise 2 Write bounded retry pseudocode

```
for each allowed attempt:
  enforce the overall deadline
  call dependency with a per-attempt timeout
  if success: return result
  if non-transient or unsafe to repeat: fail
  wait using server retry delay or backoff with jitter
  stop if delay exceeds the remaining deadline
fail with a classified error
```

Explain: retry only appropriate failures; account for SDK retries; prevent duplicate side effects with idempotency; stop on cancellation; log attempt count and final outcome. A retry policy must not turn a short user deadline into an unlimited background operation. [S8]

Exercise 3 Diagnose a deployment from evidence

Practice evidence: old pods are ready; new pods report missing configuration; the rollout has stalled; the database schema is unchanged. Expected response: stop rollout, preserve logs, restore the valid configuration or previous version, verify readiness and business behavior, then add startup configuration validation and a pipeline smoke test. Discuss how your decision changes if schema changes are incompatible.

Exercise 4 Estimate cost and capacity

Given 12,000 tasks/day, two model calls per task, and a hypothetical USD 0.006 per call, model cost is USD 144/day and USD 4,320 for a 30-day month before retries and other services. If 10% of calls repeat, model cost becomes USD 158.40/day. For capacity, 60 tasks/minute with a mean 10-second active duration implies roughly 10 concurrent tasks under stable-flow assumptions. Peak bursts, multiple calls per task, tail latency, and downstream quotas need separate headroom.

Ask yourself: What assumptions drive each result? Which metric validates them? Why might average capacity be insufficient? How would you enforce tenant fairness and avoid an unbounded queue?

Seven Day Study Plan and Mock Interview

A practical daily plan

Day 1: rehearse your introduction and two project stories. Verify dates, ownership, and CV metrics. Draw the current platform from memory.
Day 2: Azure architecture, environment boundaries, identities, networking, and recovery. Practice the whiteboard design aloud.
Day 3: APIs, idempotency, queues, database consistency, and Python or Node concurrency. Complete the run-API exercise.
Day 4: CI/CD, Terraform or Bicep, containers, probes, scaling, and rollback. Explain one actual pipeline in detail.
Day 5: telemetry and incidents. Work through all four troubleshooting scenarios and prepare a real RCA story.
Day 6: cloud and LLM cost, quotas, RAG operations, and security. Repeat the cost calculation without notes.
Day 7: run a timed mock interview, review weak answers, and prepare recruiter logistics and your questions.

If the interview is within 24 hours

Spend 30 minutes on the introduction and project evidence; 45 on Azure design and identities; 45 on CI/CD and Kubernetes; 45 on incidents and observability; 45 on cost, quotas, and agent limits; and 30 on a timed mock and recruiter questions. Take short breaks. Prioritize explanations you can defend over memorizing tool lists.

A 60 minute mock interview

Minutes 0-5: introduction and role fit. 5-15: one project and technical follow-ups. 15-30: Azure system design. 30-40: deployment and production incident. 40-48: cost and AI security. 48-55: behavior and logistics. 55-60: your questions. Have the interviewer ask why, what failed, and what you would change after each main answer.

Score each answer from zero to two

Give yourself up to two points each for directness, technical correctness, personal evidence, tradeoffs, and operational detail. A strong answer scores at least eight of ten and survives a follow-up. Record weak areas and repeat only those. For design questions, personal evidence can be a relevant prior lesson rather than a claim that you implemented the proposed design.

Final rehearsal checklist

Can I explain my two best projects without the CV? Can I distinguish message reliability from service availability? Can I explain a safe rollback and an unsafe one? Can I diagnose throttling without adding replicas? Can I calculate token cost and state the assumptions? Can I explain tenant authorization and agent tool boundaries? Can I discuss the gaps honestly and give a credible learning plan?

Quick Review and Official References

Remember these distinctions

Budget alert: spending notification; application quota: runtime enforcement. Readiness: receive traffic; liveness: restart decision; startup: protect initialization. HPA: pod count; cluster autoscaler: node capacity. Authentication: who you are; authorization: what you may do. RTO: time to restore; RPO: acceptable data loss. Retry: repeat an attempt; idempotency: protect against repeated effects. SLI: measurement; SLO: target; SLA: service agreement. Throughput, availability, and test coverage are separate measures.

Sources for deeper study

Technical references checked on 10 October 2026. Service names, pricing, quota, and supported features can vary by deployment. Use the pages below to verify the exact environment rather than memorize fixed limits. The Azure service wording in your CV describes the original project context.

S1 Microsoft Azure Well-Architected Framework — Safe deployment practices
https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/safe-deployments

S2 Microsoft Learn — Bicep what-if
https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/deploy-what-if

S3 Kubernetes documentation — Liveness readiness and startup probes
https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

S4 Microsoft Learn — Create and manage budgets
https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/tutorial-acm-create-budgets

S5 Microsoft Learn — Microsoft Entra Workload ID on AKS
https://learn.microsoft.com/en-us/azure/aks/workload-identity-overview

S6 Microsoft Learn — Azure OpenAI quota management
https://learn.microsoft.com/en-us/azure/foundry-classic/openai/how-to/quota

S7 Azure Architecture Center — AI agent orchestration patterns
https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns/

S8 Azure Architecture Center — Retry pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

Your closing answer

My strength is connecting application delivery with the operational work that keeps it dependable. I can contribute backend and Azure experience, safer deployments, practical observability, and calm incident response. I would begin by understanding your current platform and the most important reliability and cost problems, then agree on measurable improvements with the team.

Extended Technical References 1

Official references for the extended bank checked on 10 October 2026. Use these for implementation details; the answers are interview explanations rather than deployment instructions.

S9 FastAPI async and concurrency
https://fastapi.tiangolo.com/async/

S10 Node js event loop and worker pool
https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

S11 ASP NET Core performance practices
https://learn.microsoft.com/en-us/aspnet/core/fundamentals/best-practices

S12 PostgreSQL transaction isolation
https://www.postgresql.org/docs/current/transaction-iso.html

S13 OpenTelemetry signals
https://opentelemetry.io/docs/concepts/signals/

S14 Azure Service Bus dead letter queues
https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-dead-letter-queues

S15 Azure circuit breaker pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

S16 Azure Monitor log queries
https://learn.microsoft.com/en-us/azure/azure-monitor/logs/log-query-overview

Extended Technical References 2

Official references for the extended bank checked on 10 October 2026. Use these for implementation details; the answers are interview explanations rather than deployment instructions.

S17 Kubernetes Deployments
https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

S18 Azure cache aside pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

S19 OWASP SSRF prevention
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

S20 Kubernetes resource management
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

S21 Docker multi stage builds
https://docs.docker.com/build/building/multi-stage/

S22 Terraform sensitive data
https://developer.hashicorp.com/terraform/language/manage-sensitive-data

S23 Azure managed identities
https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview